Árboles

1. Breve introducción

En el vasto campo de la teoría de grafos, los árboles representan una de las estructuras más fundamentales y versátiles. Un árbol se define formalmente como un grafo conexo que no contiene ciclos. Esta aparente simplicidad estructural le otorga propiedades matemáticas únicas que son pilares en la informática y la optimización.

Entre las propiedades estructurales fundamentales se destacan aquellas que involucran su conectividad y aciclicidad. A diferencia de los grafos generales, donde pueden existir múltiples rutas entre dos puntos o incluso caminos que regresan al origen (bucles), en los árboles existe exactamente un único camino simple entre cualquier par de vértices. Si se añade una sola arista extra, se crea un ciclo; si se elimina una arista existente, el grafo deja de ser conexo. Estas propiedades las analizaremos en la Sección 2.

Además, los árboles son útiles para representar jerarquías (e.g. organigramas) mediante árboles enraizados. Aunque un árbol puede representarse de diferentes formas, a menudo se designa un vértice especial como raíz. Esto establece una jerarquía de "padres" e "hijos", permitiendo organizar la información de forma descendente. Veremos este tema en la Subsección 2.1.

Los árboles no son solo construcciones teóricas; son la columna vertebral de procesos lógicos que utilizamos a diario, como por ejemplo:

- Organización de Datos: Desde el sistema de archivos de una computadora hasta la estructura de una página web la jerarquía de árbol permite búsquedas y accesos eficientes. Un ejemplo puede verse en la figura 1.

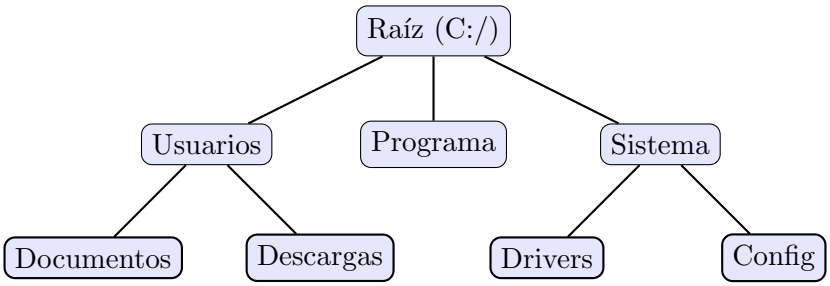


Figura 1: Ejemplo genérico de carpetas en un directorio.

- Optimización: El concepto de árbol recubridor (o de expansión, o generador) mínimo es crucial para diseñar redes (eléctricas, de transporte o de datos) que conecten todos los puntos con el menor costo posible. Estudiaremos este tópico en la Sección 3.

- Toma de decisiones: En inteligencia artificial y estadística, los árboles de decisión segmentan problemas complejos en una serie de elecciones binarias o múltiples. En la figura 2 un ejemplo muy sencillo.

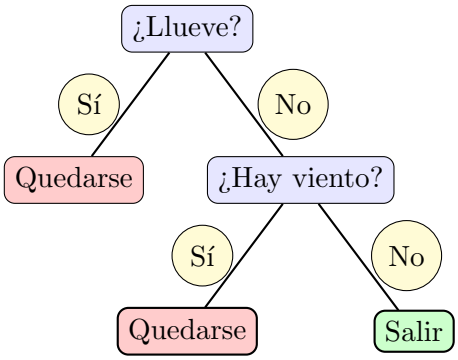


Figura 2: Un ejemplo de árbol de decisión.

2. Definiciones y propiedades básicas

Definición 2.1. Un grafo se dice *acíclico* si no tiene ciclos. Un grafo es un *bosque* si es acíclico. Un grafo es un *árbol* si es acíclico y conexo.
Una *hoja* de un árbol es un vértice del árbol de grado uno.
Un *subgrafo recubridor* (o *de expansión*) de un grafo G es un subgrafo de G con los mismos vértices de G.
Un *árbol recubridor* (o *de expansión*) de un grafo G es un subgrafo recubridor de G que es un árbol.

Ejemplos:

- P_n es un árbol.
- $K_{1,n}$ es un árbol.
- Todo árbol es bipartito.

Ejemplo 2.2. El grafo T de la figura 3 es un árbol. Vemos que T tiene cuatro hojas. Allí se muestran dos representaciones de T. En general, como veremos más adelante, es útil representar un árbol ubicando sus vértices en diferentes niveles, de forma tal que todas las aristas de T se den entre niveles consecutivos. Esto es posible dado que un árbol no tiene ciclos. Para obtener esta tipo de representación, una estrategia es seleccionar un vértice cualquiera de T para colocar en el primer nivel. Luego, poner todos sus vecinos en el siguiente nivel, y continuar de esta forma (obviamente sin repetir vértices), hasta seleccionar todos los vértices de T.

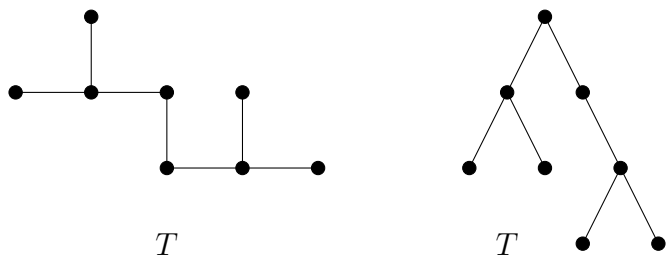


Figura 3: Dos representaciones de un árbol T.

En la figura 4 se muestra un grafo G y un árbol recubridor de G. Notar que las hojas de este árbol recubridor son a, e y f. Observar que G posee varios árboles recubridores. En particular, los

únicos grafos conexos con un único árbol recubridor son los árboles y su único árbol recubridor es el propio grafo.

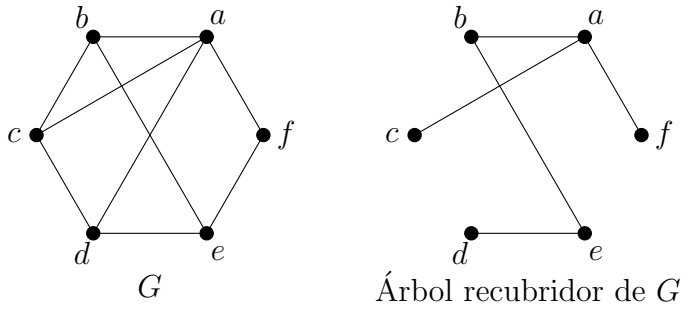


Figura 4: Grafo G y un árbol recubridor de G.

En relación a las familias de grafos conocidas, no es difícil ver que:

- P_n es un árbol para todo $n \in \mathbb{N}$. Si $n \geq 2$, P_n tiene dos hojas.
- $K_{1,n}$ es un árbol para todo $n \in \mathbb{N}$. Para $n \geq 2$, $K_{1,n}$ tiene n hojas. En general, $K_{1,n}$ tiene $\text{máx}\{2, n\}$ hojas.
- **Ejercicio**: Todo árbol es bipartito.

En todos los ejemplos anteriores de árboles, observamos que si el grafo no es trivial, tiene al menos dos hojas. En elñ siguiente resultado probamos que esto obedece a una propiedad de árboles con al menos una arista o, equivalentemente, al menos dos vértices).

Lema 2.3. Todo árbol T con al menos dos vértices tiene al menos dos hojas.

Demostración. Como T tiene al menos dos vértices, tiene al menos una arista. Sea P un camino simple maximal (que será no trivial) en T. Los extremos de P no tienen vecinos en P distintos de su único vecino en P, ya que T es acíclico.

Además, los extremos no tienen vecinos fuera de P, pues P es maximal.
Ergo, los dos extremos de P son hojas. □

Sabemos ahora que un árbol no trivial tiene hojas. Es claro que al borrar una hoja de un árbol no generaremos ciclos, por lo que el grafo continuará siendo árbol. Formalizamos esta propiedad en el siguiente resultado.

Lema 2.4. Sea T un árbol con al menos dos vértices y v una hoja de T. Entonces, $T - v$ es un árbol.

Demostración. Es claro que $T - v$ es acíclico (*) ya que T es acíclico. Además, si $u, w \in V(G) - \{v\}$, existe un u, w-camino simple P en T. Este camino no usa el vértice v, ya que $\text{gr}_T(v) = 1$ y los vértices de P distintos de u y w tienen grado al menos 2.
Luego, P es también un u, w-camino en $T - v$. En consecuencia, $T - v$ es conexo (**).

De (*) y (**), tenemos que $T - v$ es árbol. □

Puede probarse que los árboles verifican ciertas propiedades, e.g. su cantidad de aristas es una unidad menos que su cantidad de vértices, entre cualquier par de vértices existe un ún ico camino que los conecta. A continuación, no solo probaremos que estas propiedades se verifican, sino que mostraremos cómo caracterizan a los árboles.

Teorema 2.5 (Caracterizaciones)**.** Sea G un grafo con $|V(G)| = n \geq 1$, los siguientes enunciados son equivalentes.

(a) G es conexo y acíclico.

(b) G es conexo y $|E(G)| = n - 1$.

(c) G es acíclico y $|E(G)| = n - 1$.

(d) G no tiene bucles y, para cada par de vértices $u, v \in V(G)$, existe un único u, v-camino simple.

Demostración. Consideremos las siguientes 3 condiciones:

- G conexo,
- G acíclico,
- $|E(G)| = n - 1$.

Para probar la equivalencia entre (a), (b) y (C), demostraremos que dos cualesquiera de las 3 condiciones mencionadas implican la restante.

(a) $\Longrightarrow$ (b) y (c) Haremos inducción en n. Si $n = 1$,el resultado es trivial.
Supongamos que $n \geq 2$ y que vale la implicación para todo grafo con menos de n vértices. Del Lema 2.3 ($n \neq 2$) sabemos que G tiene (al menos) una hoja v y del Lema 2.4 $G - v$ es conexo y acíclico.
Como $G - v$ tiene $n - 1$ vértices,por H.I. $G - v$ tiene $n - 2$ aristas. Al agregar v agregamos exactamente una arista , resulta que $|E(G)| = n - 1$.

(b) $\Longrightarrow$ (a) y (c) Supongamos que G tiene al menos un ciclo. Borramos de G, sucesivamente, una arista de cada ciclo (i.e. borramos una arista de un ciclo y luego de este borrado, si existe algún ciclo, borramos una arista de ese ciclo y continuamos de esta forma) hasta obtener un grafo acíclico G'. Observemos que $|E(G)| > |E(G')|$ (*). Recordemos que una arista de un ciclo no es de corte (probado en la unidad anterior). Luego, G' es conexo.
Como G' es conexo y acíclico, por lo probado anteriormente, resulta que $|E(G')| = |V(G')| - 1 = |V(G)| - 1 = |E(G)|$, contradiciendo (*).
Por lo tanto, G es acíclico.

(c) $\Longrightarrow$ (a) y (b) Sea $c(G)$ la cantidad de componentes conexas de G y $G_1, \ldots, G_{c(G)}$ sus componentes conexas. Cada componente conexa es acíclica, por lo que, como hemos probado, $|V(G_i)| - 1 = |E(G_i)|$.
Por otro lado,

$$n - 1 = |E(G)| = \sum_{i=1}^{c(G)} |E(G_i)| = \sum_{i=1}^{c(G)} (|V(G_i)| - 1) = \left(\sum_{i=1}^{c(G)} |V(G_i)| \right) - c(G) = n - c(G).$$

Ergo, $c(G) = 1$, i.e. G es conexo.

Ejercicio: Probar (d)$\Longleftrightarrow$(a), (b), (c). □

Corolario 2.6. Son válidas las siguientes afirmaciones:

(a) Toda arista de un árbol es de corte.

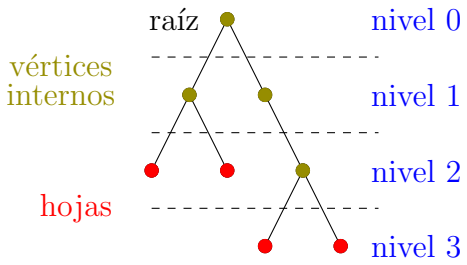
(b) Agregar exactamente una arista a un árbol (con los mismos vértices) forma exactamente un ciclo en el árbol.

(c) Todo grafo conexo tiene un árbol recubridor.

Demostración. **Ejercicio.** □

2.1. Árboles enraizados y árboles binarios

Definición 2.7. Un árbol *enraizado* T es un árbol con un único vértice distinguido r, llamado *raíz* de T. Dado $v \in V(T) - \{r\}$, sea $P(v)$ el único camino en T de r a v. El *padre* de v es el antecesor de v en $P(v)$ y los restantes vecinos de v son los *hijos* de v. Un árbol enraizado es *binario* si todo vértice tiene a lo sumo dos hijos. Si en particular todo vértice tiene 0 o 2 hijos el árbol es *binario completo*.



La *altura* del árbol enraizado es su máximo nivel.

Proposición 2.8. Si T es un árbol binario con i vértices internos entonces T tiene a lo sumo $i + 1$ hojas.

Demostración. Sea ℓ la cantidad de hojas de T.
La cantidad de vértices que tienen un padre, i.e. que son hijos, es $i - 1 + \ell$.
La cantidad de vértices que tienen hijos es i.
Como la cantidad de hijos de un vértice es a lo sumo 2, existen a lo sumo $2i$ vértices que son hijos.
Ergo, $i - 1 + \ell \leq 2i$, i.e. $\ell \leq i + 1$. □

Proposición 2.9. Si T es un árbol binario de altura $h \geq 1$ con ℓ hojas, entonces

$$\log_2(\ell) \leq h.$$

Demostración. Veremos equivalentemente que $l \leq 2^h$.
Procederemos por inducción sobre h.
Si $h = 1$, entonces $\ell \leq 2 = 2^1$ y vale la tesis.

Sea $h \geq 2$ y supongamos que el resultado es válido para todo árbol binario de altura menor que h ($h \geq 1$).
Sea T un árbol de altura h. Si la raíz r tiene un único hijo, entonces el árbol T' obtenido al borrar r y enraizarlo en el hijo de r tiene la misma cantidad de hojas ℓ que T pero tiene altura $h-1$. Por H.I. aplicada a T', $\ell \leq 2^{h-1} < 2^h$.

Si la raíz tiene dos hijos, consideramos los dos subárboles T_1 y T_2 obtenidos al borrar r y enraizarlos en los dos hijos r_1 y r_2 de r, respectivamente. Resulta entonces que la altura de T_1 (llamémosla h_1) y de T_2 (h_2) es a lo sumo $h-1$, i.e. $h_1 \leq h-1$ y $h_2 \leq h-1$.

Para $i = 1, 2$, llamamos ℓ_i a la cantidad de hojas de T que pertenecen a T_i. Tenemos que, para $i = 1, 2$, si $h_i = 0$ entonces $\ell_i = 1 = 2^{h_i} \leq 2^{h-1}$, y si $h_i \geq 1$, como $h_i \leq h-1$, aplicando HI resulta $\ell_i \leq 2_i^h \leq 2^{h-1}$.

Por lo tanto, $\ell_i \leq 2^{h-1}$ para $i = 1, 2$.
Como $\ell = \ell_1 + \ell_2$, sigue que $\ell \leq 2^{h-1} + 2^{h-1} = 2^h$. □

La noción de árbol binario puede generalizarse fácilmente como sigue.

Definición 2.10. Sea $m \geq 2$. Un árbol enraizado es *m-ario* si todo vértice tiene a lo sumo m hijos. Si en particular todo vértice tiene 0 o m hijos el árbol es *m-ario completo*.

Ejercicio: Extender las proposiciones 2.8 y 2.9 a los casos de árboles m-arios.

3. Árboles recubridores

Proposición 3.1. Sean T y T' dos árboles recubridores de un grafo conexo G y $e \in E(T) - E(T')$, entonces existe $e' \in E(T') - E(T)$ tal que $(T \setminus e) \cup e'$ es un árbol recubridor de G.

Demostración. Del Corolario 2.6, toda arista de T es de corte de T. Sean T_1 y T_2 las dos componentes conexas de $T \setminus e$.

Como T' es conexo y $(V(T_1), V(T_2))$ es una partición de $V(T')$, existe una arista $e' \in E(T')$ con un extremo en $V(T_1)$ y el otro en $V(T_2)$. Luego, $\hat{T} = (T \setminus e) \cup e'$ es conexo, tiene $n-1$ aristas y $V(\hat{T}) = V(G)$, i.e. $(T \setminus e) \cup e'$ es un árbol recubridor de G. □

Proposición 3.2. Sean T y T' dos árboles recubridores de un grafo conexo G y $e \in E(T) - E(T')$, entonces existe $e' \in E(T') - E(T)$ tal que $(T' \cup e) \setminus e'$ es un árbol recubridor de G.

Demostración. **Ejercicio.** □

Proposición 3.3. Si T es un árbol con k aristas y G es un grafo simple (no vacío) con $\delta(G) \geq k$, entonces T es subgrafo (no necesariamente inducido) de G.

Demostración. Haremos inducción sobre k. Si $k = 0$,el resultado es directo ya que K_1 es subgrafo de todo grafo no vacío.

Sea $k \geq 1$ y supongamos que el resultado es válido para árboles con menos de k aristas.

Por los lemas 2.3 y 2.4, tenemos que existe $v$hoja de T y que $T' = T - v$ es un árbol. Sea u el padre (único vecino) de $v$en T. Como $|E(T')| = k-1$ y $\delta(G) \geq k > k-1$, por H.I. T' es subgrafo

de G.
Observemos que $|N_G(v)| = \mathrm{d}_G(v) \geq k$, $v \notin N_G(v)$ (G simple) y T' tiene k vértices, entonces existe en G un vértice w vecino de v que no pertenece a T'.
Luego, agregando a T' el vértice w y la arista vw, tenemos a T como subgrafo de G. □

Observación 3.4.

- Observemos que la cota inferior para el grado mínimo de G en la Proposición 3.3, i.e. $\delta(G) \geq k$, es ajustada. Dicho de otra forma, el resultado no es válido en general si reemplazamos esa cota por $\delta(G) \geq k-1$, ya que, e.g., K_k tiene $\delta(K_k) = k-1$ y no contiene ningún árbol con k aristas (pues todo árbol con k aristas debe tener $k+1$ vértices).

- Existen diferentes estrategias para hallar árboles recubridores en grafos conexos. Dos clásicas son:

  - **Búsqueda en anchura (Breadth-First Search, BFS)**: Mostramos dos algoritmos en los cuales se puede obtiene una función predecesor, lo que da lugar a un árbol recubridor de G. Además, se calcula la distancia de u a los restantes vértices. En el Algoritmo 1 no hay orden preestablecido de vértices y en el Algoritmo 2 tenemos como entrada un orden de preferencias. Además,en la figura 6 mostramos el árbol obtenido como árbol enraizado en u donde p indica el padre de cada vértice (salvo de $u = a$).

  - **Búsqueda en profundidad (Depth-First Search, DFS)**: Análogo a lo anterior, en el Algoritmo 3 mostramos una función predecesor y el árbol obtenido. En las figuras 5 y 6 mostramos el árbol obtenido con el orden alfabético. En particular en la figura 6 lo mostramos como árbol enraizado en $u = a$ Dejamos como **ejercicio** escribir el algoritmo DFS para el caso en el cual tenemos un orden de preferencia de los vértices.

Algorithm 1: BFS (G, p, d)

Input : G grafo conexo y $u \in V(G)$.
Output: Una función predecesor p, y una función distancia $d(u, v)$ para todo $v \in V(G)$.

```
Q := (u), R = {u}, d(u,u) = 0;
while R ≠ V(G) do
    considerar x el primer vértice de Q;
    while existe y ∈ N_G(x) \ R do
        agregar y atrás en Q;
        R = R ∪ {y};
        p(y) = x;
        d(u,y) = d(u,x) + 1;
    quitar x de Q;
mostrar (p, d);
```

En la figura 5 mostramos un grafo G y los árboles T_B y T_D obtenidos mediante BFS y DFS, respectivamente, con el orden alfabético de los vértices. Estos árboles se construyen a partir de la función p obtenida mediante los algoritmos. Por ejemplo, en el algoritmo BFS, tenemos $a = p(b) = p(d) = p(i)$, $b = p(h)$, $h = p(g)$ y $d = p(c) = p(f)$. Además, se obtiene la función distancia d de a a los vértices de G como sigue: $0 = d(a, a)$, $1 = d(a, b) = d(a, d) = d(a, i)$, $2 = d(a, h) = d(a, c) = d(a, f)$ y $3 = d(a, g)$.

Algorithm 2: BFS $(G, (v_1, \ldots, v_n), p, d)$

Input : G grafo conexo y un orden de sus vértices $v_1, \ldots, v_n$.
Output: Una función predecesor p, y una función distancia $d(v_1, v_i)$ para todo $i \in [n]$.

```
Q := (v_1), R = {v_1}, d(v_1, v_1) = 0;
while R ≠ V(G) do
    considerar x el primer vértice de Q;
    for i = 2 to n do
        if v_i ∈ N_G(x) \ Q then
            agregar v_i atrás en Q;
            R = R ∪ {v_i};
            p(v_i) = x;
            d(v_1, v_i) = d(v_1, x) + 1;
    quitar x de Q;
mostrar (p, d);
```

Algorithm 3: DFS (G, p)

Input : G grafo conexo y $u \in V(G)$.
Output: Una función predecesor p.

```
Q := (u), R = {u};
while R ≠ V(G) do
    Considerar x el primer vértice de Q;
    if existe y ∈ N_G(x) \ R then
        agregar y adelante (primero) en Q;
        R := R ∪ {y};
        p(y) = x;
    else
        quitar x de Q;
mostrar p;
```

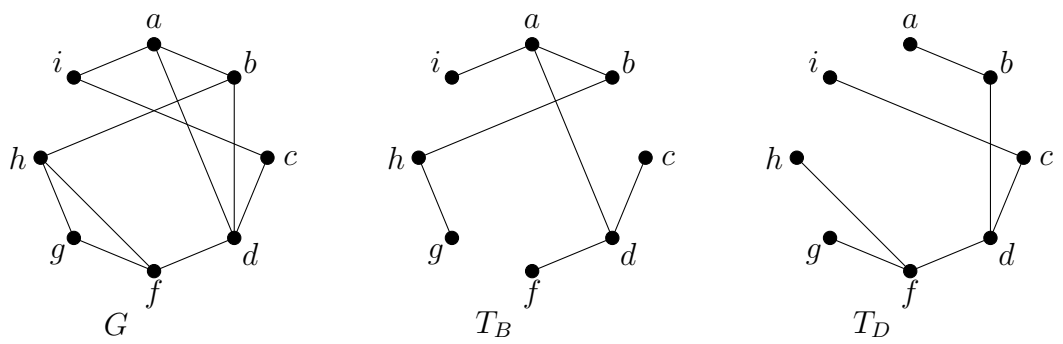


Figura 5: Grafo G y dos árboles de recubridores T_B y T_D obtenidos mediante BFS y DFS, respectivamente. Consideramos el orden alfabético de los vértices.

3.1. Árboles recubridores de costo óptimo (mínimo o máximo)

En modelizaciones mediante grafos de muchos problemas, las aristas tienen asignado un peso o costo, el cual puede provenir de que la arista representa una ruta o una conexión entre servidores,

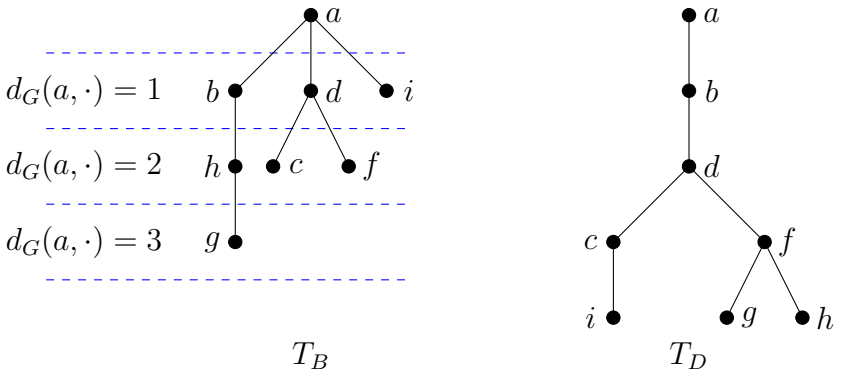


Figura 6: Árboles de recubridores T_B y T_D de la figura 5, graficados enraizados en $u = a$. Observemos que en el caso de T_B, se evidencia la función distancia en cada nivel.

y tiene asociado un costo por el combustible y el peaje para usar esa ruta o la cantidad de cable a utilizar para la conexión representada por la arista. Entonces, usar diferentes aristas implica distintos costos.
Dado un grafo conexo G, todo árbol recubridor de G utiliza la misma cantidad de aristas, pero si estas tienen costos, al sumar los costos de todas las aristas de cada árbol recubridor, estos pueden ser diferentes. Es por esto que veremos una forma de obtener árboles recubridores de grafos conexos con pesos asociados a sus arisztas, tales que esos árboles tienen costo óptimo (mínimo o máximo).

Ejemplo 3.5. Consideremos se necesita realizar una red entre las ciudades argentinas del cuadro1, de forma tal que todas las ciudades queden conectadas. El costo de realizar una conexión es proporcional a la distancia entre las ciudades. Podemos modelizar esta situación mediante un grafo G cuyo conjunto de vértices es el conjunto de las ciudades mencionadas, y posee aristas entre todo par de ciudades con un costo asociado a cada una igual a la distancia entre el par de ciudades que une.
Luego, como todos los costos son positivos, para mantener conectadas estas ciudades a mínimo costo, usaremos una estructura de árbol recubridor. Esto surge por el hecho de que para mantener conectados estos 9 vértices, sabemos que necesitamos como mínimo 8 aristas. Además, si tenemos un subgrafo recubridor con 9 o más aristas, también conocemos que contiene al menos un ciclo, por lo que tiene una arista que no lo desconecta (no es de corte) y, como esa arista tendrá costo positivo, al eliminarla tendremos un subgrafo recubridor de peso menor.
En definitiva, buscamos un árbol recubridor de G de peso mínimo. Una idea sencilla, es seleccionar de manera sucesiva aristas de peso mínimo de forma tal que no se generen ciclos. Observar que al seleccionar 8 aristas distintas con la condición de no generar ciclos, tendremos un árbol recubridor de G. En la tabla 1 figuran en rojo las aristas seleccionadas y el orden en el cual fueron elegidas. En la figura 7 graficamos el árbol recubridor con estas aristas. El peso total es 2212.
Pero, ¿este árbol recubridor es de peso mínimo entre todos los árboles recubridores?
Formalizaremos esta idea en el *Algoritmo de Kruskal* y probaremos que es efectiva para nuestro objetivo.

A continuación formalizaremos las nociones anteriores. En particular, definiremos grafo ponderado y mostraremos el Algoritmo de Kruskal para hallar un árbol recubridor de peso mínimo en un grafo conexo (incluso si hay pesos negativos).

Definición 3.6. Un *grafo ponderado* es un grafo con etiquetas numéricas en sus aristas y/o

	Bs As	MdP	BB	C.	P.	S.R.	L.P.	GG
Rosario	279	627	657	375(7ma)	135(2da)	623	331	260(4ta)
Bs As		383	573	648	377	612	52(1ra)	230(3ra)
MdP			464(8va)	1110	913	729	369(6ta)	644
BB				937	974	327(5ta)	631	850
Córdoba					395	612	754	654
Paraná						818	551	288
Santa Rosa							666	747
La Plata								289

Cuadro 1: Distancias entre ciudades. Quizás vale aclarar: GG: Gualeguaychú, BB: Bahía Blanca, MdP: Mar del Plata.

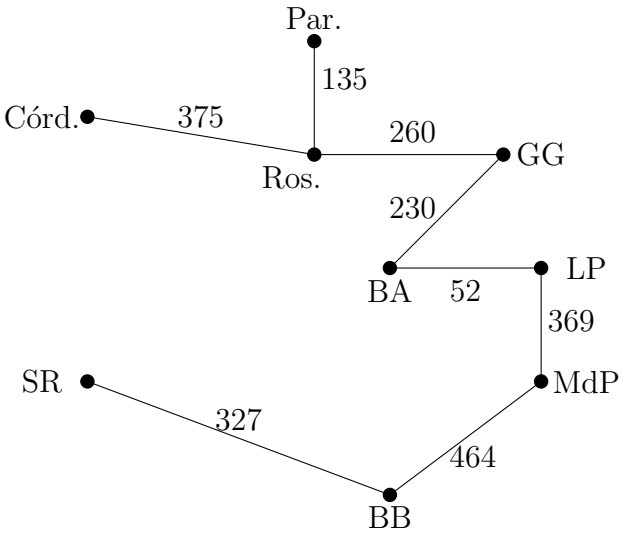


Figura 7: Árbol recubridor obtenido mediante el procedimiento descripto con las aristas seleccionadas como indica la tabla 1.

vértices. En particular, en esta sección usaremos etiquetas solo en las aristas.

Algoritmo de Kruskal (para mínimo)

Entrada: Un grafo conexo G.

Idea: Mantener un subgrafo recubridor acíclico H e ir agregando aristas de peso mínimo.

Inicio: $E(H) = \emptyset$, $V(H) = V(G)$.

Iteración: Mientras existan aristas que unan dos componentes conexas de H, agregar la de mínimo peso a $E(H)$.

En el Teorema 3.8 probaremos que efectivamente el Alg. de Kruskal devuelve un árbol recubridor de G de peso mínimo. Vemos ahora otro ejemplo de aplicación de este algoritmo.

Ejemplo 3.7. Consideremos el grafo G de la figura 8. Al inicio $V(H) = \{a, b, c, d, e, f\}$ y $E(H) = \emptyset$. En esta instancia todas las aristas unen componentes conexas diferentes. Seleccionamos una de menor peso, tomemos ac de peso 1 e incorporamos esta arista a H. Luego, nuevamente todas las aristas restantes unen dos componentes conexas, una (en este caso la única) de peso mínimo es de de peso 1. Incorporamos esta arista a H y continuamos. La próxima arista a incorporar en H es ab, luego be. Entre las restantes aristas que no pertenecen a H a esta altura, la de peso mínimo es ad, pero esta arista no une componentes conexas diferentes de H (i.e. de agregarla formaría un ciclo en H). El algoritmo selecciona entonces la arista af. Ahora H es conexo, por lo que no hay aristas que unan dos componentes conexas de H, el alg. culmina y resylta H un árbol recubridor de G de peso mínimo.

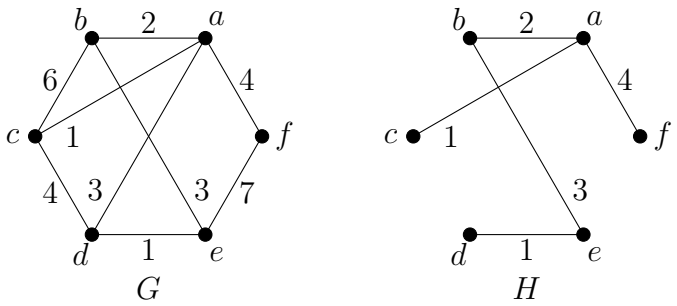


Figura 8: Grafo G y un árbol recubridor de peso mínimo H obtenido mediante el Alg. de Kruskal.

Llamaremos w a la función que asigna pesos en las aristas de un grafo G. Además, dado G' subgrafo de G, notaremos $w(G') = \sum_{e \in E(G')} w(e)$.

Teorema 3.8. Si G es un grafo conexo ponderado, el Algoritmo de Kruskal produce un árbol recubridor de G de peso mínimo.

Demostración. Observemos primero que H es un árbol, ya que al agregar aristas entre diferentes componentes conexas no se forman ciclos. Además, H es conexo, ya que si existieran dos o más componentes conexas en H, entonces no existen aristas de G entre esas componentes conexas y entonces G sería disconexo. Por lo tanto H es un árbol.

Sea H' un árbol recubridor de G de peso mínimo. Veamos que $w(H) = w(H')$.

Supongamos que esto último no es cierto. Luego, $w(H') < w(H)$. Tenemos entonces que $H' \neq H$ y en consecuencia $E(H) - E(H') \neq \emptyset$.

Sea e la primera arista de $E(H) - E(H')$ seleccionada por el algoritmo. De la Proposición 3.2 tenemos que existe una arista $e' \in E(H') - E(H)$ tal que $(H' \cup \{e\}) \setminus e'$ es un árbol recubridor de G.

Como e' y (si las hay) todas las aristas de H seleccionadas por el algoritmo antes de e pertenecen a H', tanto e como e' están disponibles (i.e. unen dos componentes conexas) cuando el algoritmo selecciona la arista e. Ergo, $w(e) \leq w(e')$.

Por lo tanto, $w((H' \cup \{e\}) \setminus e') \leq w(H')$. En consecuencia, $H'' = (H' \cup \{e\}) \setminus e'$ es un árbol recubridor de G de peso mínimo. Además, H'' coincide con H en más aristas de H'.

Continuando con este razonamiento (y dada la finitud de la cantidad de aristas) obtenemos un árbol recubridor de G de peso mínimo que coincide con H. Por lo tanto, H es de peso mínimo. $\square$

Observación 3.9. Para finalizar esta sección mencionamos que, si bien trabajamos considerando siempre árboles recubridores de peso mínimo, se pueden obtener resultados análogos considerando árboles recubridores de peso máximo y cambiando en el Alg. de Kruskal mínimo por máximo.
También puede reducirse un problema al otro. Por ejemplo, si G es un grafo conexo ponderado y nos interesa obtener un árbol de peso máximo, podemos considerar un grafo G' cambiando los pesos en las aristas y luego buscar un árbol recubridor de peso mínimo de G'. Este, será de peso máximo en G, con los pesos originales.
Un posible cambio en los pesos es el siguiente: sea $M = \text{máx}\{w(e) : e \in E(G)\}$, definimos $w'(e) = M - w(e)$ los pesos en G'. Luego, si H' es un árbol recubridor de peso mínimo de G', H' también será árbol recubridor pero de peso máximo de G y ese peso máximo será $w(H') = M(|V(G)| - 1) - w'(H')$.

Ejercicio: ¿Funciona como en el caso anterior si los pesos de G' los definimos como los opuestos de los pesos en G?

Observación 3.10. Otro de los algoritmos clásicos para hallar árboles recubridores óptimos de grafos conexos es el Algoritmo de Prim, con el cual trabajeros en práctica.

4. Algoritmo de Dijkstra: Distancia mínima de un vértice a los restantes

Hemos visto que, dado un grafo conexo y fijado uno de sus vértices u, mediante un BFS podemos hallar la distancia de u a los restantes vértices de G. Es decir, hallamos un camino más corto entre u y los demás vértices. En este caso, cada arista cuenta como una unidad para la distancia entre cada par de vértices. En el caso de grafos ponderados, la longitud de un camino es la suma de los pesos de sus aristas. Veremos aquí uno de los algoritmos más importantes y clásicos de la Teoría de Grafos para hallar la distancia mínima entre un vértice dado y los restantes.

Algoritmo de Dijkstra

Entrada: Un grafo G ponderado con pesos no negativos y un vértice de inicio u. El peso de una arista xy es $w(xy)$ y si $xy \notin E(G)$, consideramos $w(xy) = \infty$.

Idea: Considerar un conj. S de vértices para los cuales hallamos un camino mínimo desde u, "agrandando" S hasta incluir todos los vértices. Tendremos una distancia arbitraria $t(z)$ desde u a cada $z \notin S$, hasta que la distancia mínima sea hallada.

Inicio: $S = \{u\}$, $t(u) = 0$, $t(z) = w(uz), \forall z \neq u$.

Iteración: Considerar $v \notin S$ tal que $t(v) = \text{mín}\{t(z) : \ z \notin S\}$.
Agregar v a S.
Explorar las aristas desde v y actualizar las etiquetas $t(z)$ para z vecino de v y $z \notin S$ con $t(z) = \text{mín}\{t(z), t(v) + w(vz)\}$. Es decir, para cada $z \in N_G(v) \setminus S$, $t(z) := \text{mín}\{t(z), t(v) + w(vz)\}$.

Continuar la iteración hasta que $S = V(G)$ o hasta que $t(z) = \infty \ \forall z \notin S$.
Finalmente, considerar $d(u, v) = t(v), \forall v \in V(G)$.

Ejemplo 4.1. Consideremos el grafo G de la figura 9 y apliquemos el Alg. de Dijkstra. Tenemos:

1: Inicio: $S = \{u\}$, $t(u) = 0$, $t(a) = 1$, $t(b) = 3$, $t(c) = t(d) = t(f) = \infty$.

2: Tenemos que $\text{mín}\{t(z) : \ z \notin S\} = 1 = t(a)$.

3: Agregamos a a S, $S = \{u, a\}$.

4: $N_G(a) \setminus S = \{c, d\}$. Actualizamos las etiquetas $t(c) := \text{mín}\{t(c), t(a) + w(ac)\} = \text{mín}\{\infty, 1 + 5\} = 6$ y $t(d) := \text{mín}\{t(d), t(a) + w(ad)\} = \text{mín}\{\infty, 1 + 4\} = 5$.

5: Resulta, para los vértices no pertenecientes a S, $t(b) = 3$, $t(c) = 6$, $t(d) = 5$, $t(f) = \infty$.

6: Agregamos b a S, $S = \{u, a, b\}$.

7: $N_G(b) \setminus S = \{c, d\}$. Actualizamos las etiquetas $t(c) := \text{mín}\{t(c), t(b) + w(bc)\} = \text{mín}\{6, 3+4\} = 6$ y $t(d) := \text{mín}\{t(d), t(b) + w(bd)\} = \text{mín}\{5, 3 + 1\} = 4$.

8: Resulta, para los vértices no pertenecientes a S, $t(c) = 6$, $t(d) = 4$, $t(f) = \infty$.

9: Agregamos d a S, $S = \{u, a, b, d\}$.

10: $N_G(d) \setminus S = \{f\}$. Actualizamos la etiqueta $t(f) := \text{mín}\{t(f), t(d)+w(df)\} = \text{mín}\{\infty, 4+6\} = 10$.

11: Resulta, para los vértices no pertenecientes a S, $t(c) = 6$, $t(f) = 10$.

12: Agregamos c a S, $S = \{u, a, b, d, c\}$.

13: $N_G(c) \setminus S = \{f\}$. Actualizamos la etiqueta $t(f) := \text{mín}\{t(f), t(c)+w(cf)\} = \text{mín}\{10, 6+2\} = 8$.

14: Resulta, para los vértices no pertenecientes a S, $t(f) = 8$.

15: Agregamos f a S, $S = \{u, a, b, d, c, f\}$.

16: $N_G(f) \setminus S = \emptyset$.

17: Como $S = V(G)$, no volvemos a iterar.

18: Salida: Para cada $v \in V(G)$, $d(u, v) = t(u)$. Es decir, $d(u, u) = 0$, $d(u, a) = 1$, $d(u, b) = 3$, $d(u, d) = 4$, $d(u, c) = 6$, $d(u, f) = 8$.

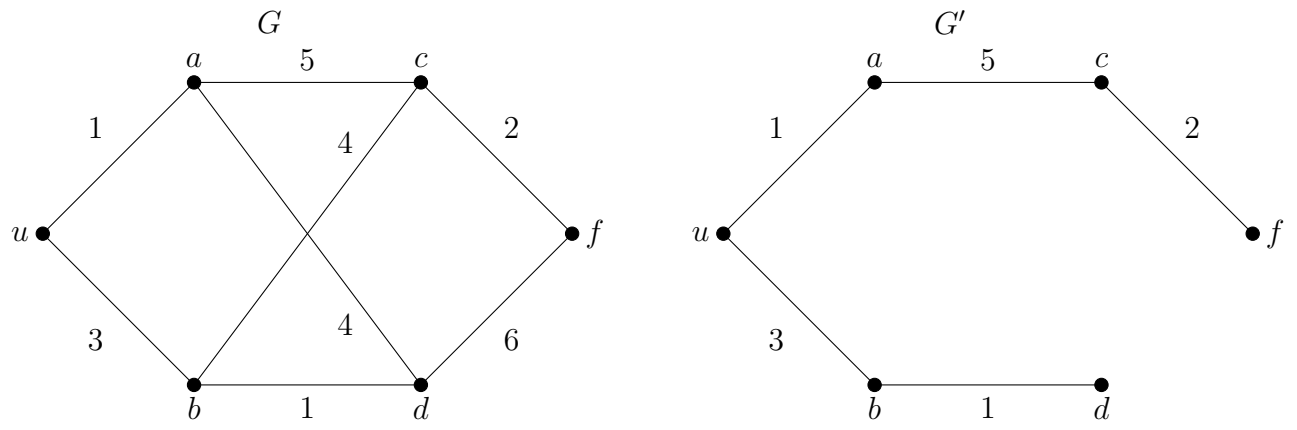


Figura 9: Grafo ponderado G. En el grafo G' dejamos solamente las aristas de los caminos mínimos desde u a los restantes vértices.

Otra opción para mostrar la aplicación del alg. de Dijkstra es colocar las etiquetas directamente en el gráfico.

Además, podemos obtener los caminos más cortos guardando, para cada vértice, el dato del vértice inmediato anterior desde el cual se obtiene la etiqueta mínima. En la figura 10 realizamos un ejemplo para lo visto en el grafo ponderado G de la figura 9. En la práctica queda como ejercicio la modificación formal en el algoritmo para mostrar los caminos más cortos.

Teorema 4.2. Dado un grafo ponderado G y un vértice $u \in V(G)$, el Alg. de Dijkstra calcula la distancia $d(u, z)$ de u a todo $z \in V(G)$.

Demostración. Veamos que se verifica lo siguiente en cada iteración:

(a) para $z \in S$, $t(z) = d(u, z)$,

(b) para $z \notin S$, $t(z)$ es la menor longitud de un u, z-camino que llega a z por una arista desde S.

Haremos inducción sobre $|S| = k$. Si $k = 1$, $S = \{u\}$, $d(u, u) = t(u) = 0$ y si $z \notin S$ entonces la mínima longitud de un u, z-camino desde $S = \{u\}$ es $t(z) = w(u, z)$.

Supongamos que valen (a) y (b) para $|S| = k$ y sea $v \notin S$ tal que $t(v) = \text{mín}\{t(z) : \ z \notin S\}$. El algoritmo selecciona el vértice v. Sea $S' = S \cup \{v\}$. Veamos que para S' se verifican (a) y (b).

Si $z \in S' \setminus \{v\} = S$, por H.I. tenemos $t(z) = d(u, z)$. Ergo, para probar (a) para S' resta ver que $d(u, v) = t(v)$. Por H.I. ítem (b), la longitud de un u, v-camino mínimo que llega a v con una arista desde S es $t(v)$. Como todo u, v-camino debe salir de S y llegar a v, si pasa por un vértice $z \notin S$ con $z \neq v$, como $t(z) \geq t(v)$, este camino tendrá al menos la longitud del u, v-camino que llega a v por una arista desde S.

Ergo, $d(u, v) = t(v)$ y vale (a) para S'.

Probemos ahora (b) para S'. Sea $z \notin S'$, i.e. $z \notin S$ y $z \neq v$. Queremos probar que $\text{mín}\{t(z), t(v) + w(v, z)\}$ es la mínima longitud entre los u, z-caminos que llegan a z desde una arista de S'.

Por H.I., la mínima longitud de estos caminos que llegan a z desde una arista de S es $t(z)$. Como agregamos v a S y se actualiza la etiqueta a $\text{mín}\{t(z), t(v) + w(v, z)\}$, estamos considerando también los u, z-caminos que llegan a z por una arista desde v. Finalmente, observemos que la longitud mínima de un camino que llega a z desde v es $t(v) + w(v, z)$. Esto prueba (b) para S'. $\square$

Observación 4.3. El Alg. de Dijkstra puede adaptarse sin dificultades para grafos ponderados dirigidos, respetando las direcciones de los arcos. Es decir, si tenemos un grafo dirigido ponderado y buscamos el camino dirigido mínimo entre un nodo fijo u y los restantes, podemos utilizar el Alg. de Dijkstra con la adaptación que imponen las direcciones de los arcos. Veremos más sobre esto en la práctica.

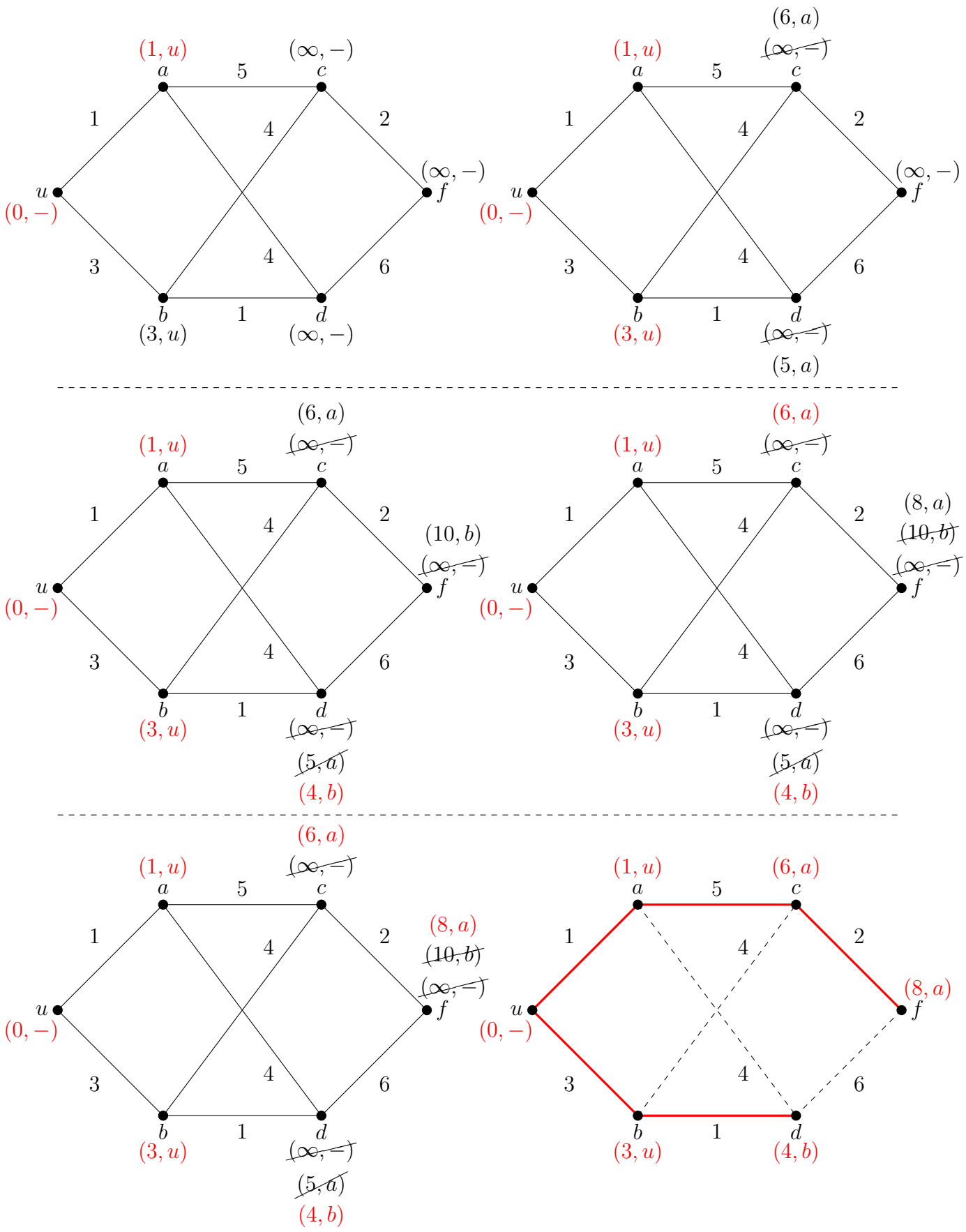


Figura 10: Algoritmo de Dijkstra plasmado gráficamente.